

19.04.2026



Yıldız Teknik Üniversitesi
Elektrik-Elektronik Fakültesi
Bilgisayar Mühendisliği Bölümü

BLM2022 Bilgisayar Organizasyonu

Ödev: 1

Gr: 2

Doç.Dr. Ali Can KARACA

İsim: Nasrullah Toprak

Öğrenci No: 24011949

E-posta: nasrullah.toprak@std.yildiz.edu.tr

1. Amaç:

Bu ödevin amacı, temel mantık kapıları kullanılarak yapısal seviyede bir RISC-V ALU tasarlamak ve bu ALU'yu bir register file ile birleştirerek çalışan bir veri yolu sistemi oluşturmaktır. Bu kapsamda ALU'nun istenilen işlemleri doğru şekilde gerçekleştirmesi sağlanmış, datapath yapısı kurulmuş ve sistemin zamanlama analizi yapılmıştır. Ayrıca maksimum çalışma frekansı hesaplanmış, hold-time ihlalleri kontrol edilerek gerekli durumlarda tasarım iyileştirmeleri gerçekleştirilmiştir.

2. ALU Mimarisi:

Bu çalışmada 32-bit genişliğinde bir ALU (Arithmetic Logic Unit) tasarlanmıştır. ALU'nun amacı, iki adet 32-bit giriş verisi üzerinde seçilen aritmetik veya mantıksal işlemi gerçekleştirerek sonucu üretmektir.

Giriş ve Çıkışlar

- Girişler:
 - A[31:0]: Birinci operand
 - B[31:0]: İkinci operand
 - ALUControl[2:0]: Yapılacak işlemi belirleyen kontrol sinyali
- Çıkışlar:
 - Result[31:0]: İşlem sonucu
 - Zero: Sonucun sıfır olup olmadığını gösterir
 - Overflow: Aritmetik işlemler sırasında taşma (overflow) durumunu gösterir

Desteklenen İşlemler

ALUControl	İşlem
000	AND
001	XOR
010	ADD
110	SUB
111	SLT

Mantıksal İşlemler (AND ve XOR)

- AND işlemi için her bit: $\text{Result}[i] = A[i] \& B[i]$
- XOR işlemi için her bit: $\text{Result}[i] = A[i] \oplus B[i]$

Bu işlemler tamamen paralel şekilde 32 bit boyunca uygulanmaktadır.

Aritmetik İşlemler (ADD [010] ve SUB [110])

Toplama ve çıkarma işlemleri aynı donanım altyapısı kullanılarak gerçekleştirilmiştir.

Bu amaçla:

- ALUControl sinyalinin en anlamlı biti (ALUControl[2]) kullanılmıştır.
- Bu bit:
 - 0 ise $\rightarrow$ ADD
 - 1 ise $\rightarrow$ SUB

Çıkarma işlemi doğrudan yapılmamış, bunun yerine ikiye tümleyen (2's complement) yöntemi kullanılmıştır: $A - B = A + (\bar{B} + 1)$

Bu doğrultuda:

B girdisi, her bit için: $B_{arimetik}[i] = B[i] \oplus ALUControl[2]$ şeklinde hazırlanmıştır.

- ADD durumunda: B aynen kullanılır
- SUB durumunda: B terslenir

Aynı kontrol biti, adder girişine carry-in (C_{in}) olarak verilmiştir:

- ADD için: $C_{in} = 0$
- SUB için: $C_{in} = 1$

Bu sayede tek bir toplama devresi ile hem ADD hem de SUB işlemleri gerçekleştirilmiştir.

SLT (Set Less Than) İşlemi

SLT işlemi, A ve B sayılarının işaretli karşılaştırmasını yapar. Bu işlem:

- Öncelikle $A - B$ işlemi gerçekleştirilerek yapılır
- Sonucun işaret biti ve taşma (overflow) durumu birlikte değerlendirilir

Karşılaştırma sonucu:

- Eğer $A < B$ ise $\rightarrow$ sonuç 1
- Aksi halde $\rightarrow$ sonuç 0

Bu sonuç yalnızca en düşük anlamlı bite (LSB) yazılır, diğer bitler sıfır yapılır.

Sonuç Seçimi (Multiplexer Yapısı)

ALU içerisinde farklı işlemler paralel olarak hesaplanmaktadır:

- AND sonucu
- XOR sonucu
- Aritmetik (ADD/SUB) sonucu
- SLT sonucu

Bu sonuçlar, ALUControl sinyaline bağlı olarak çoklayıcılar (MUX) aracılığıyla seçilerek nihai Result çıkışı oluşturulur.

Zero Bayrağı

Zero çıkışı, ALU sonucunun sıfır olup olmadığını gösterir.

Bu amaçla:

- Tüm bitler OR işlemi ile birleştirilir
- Sonuç terslenerek Zero sinyali elde edilir

$Zero = 1$ eğer $Result = 0$

Overflow Bayrağı

Overflow yalnızca aritmetik işlemler (ADD, SUB, SLT) için anlamlıdır.

Bu durum: en anlamlı bitin (MSB) giriş ve çıkış taşıma değerleri karşılaştırılarak belirlenir

Mantık: $Overflow = C_{31} \oplus C_{32}$ Bu değer, yalnızca aritmetik işlemler seçildiğinde çıkışa verilmiştir.

3. Adder Seçimi ve Gerekçesi:

Bu tasarımda toplama/çıkarma işlemleri için Carry Lookahead Adder (CLA) tercih edilmiştir.

Basit bir Ripple Carry Adder (RCA) yapısında taşıma (carry) sinyali, her bitten bir sonraki bite ardışık (zincirleme) olarak iletilir. Bu durumda: $t_{RCA} = N t_{Full_Adder}$ yani bit sayısı arttıkça gecikme lineer olarak büyür. Özellikle 32-bit gibi geniş yapılarda bu durum kritik yolun uzamasına ve maksimum frekansın düşmesine neden olur.

Buna karşılık CLA yapısında taşıma sinyalleri: propagate (P) ve generate (G) sinyalleri kullanılarak önceden hesaplanır.

Bu sayede taşıma sinyali: tüm bitler boyunca sırayla yayılmak yerine paralel mantık yapıları ile daha hızlı elde edilir

Dolayısıyla gecikme: $t_{CLA} = t_{pg} + t_{pg_block} + (N/k - 1)t_{AND_OR} + k t_{FA}$ olur ve özellikle büyük bit genişliklerinde önemli hız avantajı sağlar.

4. Fmax matematiksel hesabı:

Bu bölümde sistemin maksimum çalışma frekansı hesaplanmıştır. Bunun için önce setup-time kısıtı kullanılarak minimum clock periyodu (T_c) bulunur, ardından F_{max} hesaplanır.

$$T_c \geq t_{pcq} + t_{pd} + t_{setup}$$

t_{pcq} = kaynak register'ın clock-to-Q maksimum gecikmesi

t_{pd} = iki register arasındaki kombinasyonel mantığın maksimum gecikmesi

t_{setup} = hedef register'ın setup time değeri

Bizim tasarımımızda bu ifade şu şekilde uygulanır:

$$(t_{pd} = t_{pd,read} + t_{pd,ALU} + t_{pd,wd_mux})$$

$$T_c \geq t_{pcq} + t_{pd}(read + ALU + wd_mux) + t_{setup}$$

Bilinen gecikmeler

regfile.v dosyasından:

$$t_{pcq} = 22 \text{ ps}$$

$$t_{setup} = 20 \text{ ps}$$

$$t_{pd,read} = 25 \text{ ps}$$

lib_gates.v dosyasından maksimum gecikmeler:

$$t_{pd}(AND) = 25 \text{ ps}$$

$$t_{pd}(OR) = 25 \text{ ps}$$

$$t_{pd}(XOR) = 40 \text{ ps}$$

$$t_{pd}(MUX) = 30 \text{ ps}$$

Datapath içindeki son yazma MUX'u için:

$$t_{pd,wd_mux} = 30 \text{ ps}$$

ALU içindeki en uzun yolun belirlenmesi:

ALU içinde en uzun yol aritmetik yol üzerinden oluşur. En kritik yol, carry üretimi ve sonuç seçme mantığını içerdiği için ADD/SUB/SLT tarafındadır. Ödev metninde de maksimum frekansı belirleyen bloğun adder/subtractor olduğu açıkça belirtilmiştir. Bu tasarımda en uzun ALU yolu SLT işlemi üzerinden hesaplanmıştır. Çünkü bu işlemde önce subtraction yolu çalışmakta, ardından overflow hesaplanmakta ve son olarak SLT sonucu üretilmektedir.

tpd,ALU hesabı

$b_arith[i] = B[i] \text{ xor } ALUControl[2] \rightarrow$ Bu yol 1 adet XOR içerir: $\rightarrow 40 \text{ ps}$

İlk 4-bit CLA bloktan carry çıkışı: $\rightarrow$ İlk bloktan c4 oluşumu en kötü durumda $\rightarrow 165 \text{ ps}$

Bloklar arası carry yayılması İlk bloktan sonra 6 adet blok daha vardır ve her blokta carry yolu yaklaşık olarak: $\rightarrow 1 \text{ AND} + 1 \text{ OR} = 25 + 25 = 50 \text{ ps}$ olur. Bu nedenle: $6 \times 50 = 300 \text{ ps}$

Dolayısıyla: $c28 = 165 + 300 = 465 \text{ ps}$

Son blok içinde bit31 için carry-in oluşumu: Son blokta c28'den c31_internal'a ulaşmak için: 50 ps

Dolayısıyla: $c31_internal = 465 + 50 = 515 \text{ ps}$

C31 ve Cout çıkışlarının buffer ile alınması: Kodda bu çıkışlar OR buffer ile dışarı verildiği için:

$c31 = 515 + 25 = 540 \text{ ps}$

$c32 = 515 + 25 = 540 \text{ ps}$

Overflow hesabı

Overflow = C31 xor C32 $\rightarrow$ overflow = $540 + 40 = 580 \text{ ps}$

Sum[31] hesabı

$sum[31] = x3 \text{ xor } c31_internal$: burada baskın olan yol c31_internal olduğundan:

$sum[31] = 515 + 40 = 555 \text{ ps}$

set_less hesabı

$set_less = sum[31] \text{ xor } overflow$: $set_less = \max(555, 580) + 40$

$set_less = 580 + 40$

$set_less = 620 \text{ ps}$

slt_out[0] üretimi

slt_out[0] için 1 adet OR vardır:

$slt_out[0] = 620 + 25 = 645 \text{ ps}$

Sonuç seçme MUX'ları

SLT sonucunun nihai Result çıkışına ulaşması için 2 adet MUX geçilir:

$645 + 30 + 30 = 705 \text{ ps}$

Böylece: $tpd,ALU = 705 \text{ ps}$

Tc hesabı

$$T_c \geq t_{pcq} + t_{pd}(read + ALU + wd_mux) + t_{setup}$$

$$T_c \geq 22 + 25 + 705 + 30 + 20$$

$$T_c \geq 802 \text{ ps} \rightarrow T_c(\min) = 802 \text{ ps}$$

Fmax hesabı

$$F_{max} = 1 / T_c = 1 / (802 \text{ ps}) = 1.247 \text{ GHz}$$

5. Hold-time analizi:

$$t_{ccq} + t_{cd} > t_{hold}$$

Bizim devrede:

$$t_{cd} = t_{cd,read} + t_{cd,ALU} + t_{cd,mux}$$

Bilinen değerler:

$$\begin{aligned} t_{ccq} &= 10 \text{ ps} \\ t_{hold} &= 60 \text{ ps} \\ t_{cd,read} &= 12 \text{ ps} \\ t_{cd}(AND) &= 15 \text{ ps} \\ t_{cd}(MUX) &= 15 \text{ ps} \end{aligned}$$

ALU içindeki en kısa yol:

$$\begin{aligned} t_{cd,ALU} &= t_{cd}(AND) + 3 \cdot t_{cd}(MUX) \\ &= 15 + 3 \cdot 15 = 60 \text{ ps} \end{aligned}$$

Toplam kısa yol:

$$t_{cd} = t_{cd,read} + t_{cd,ALU} + t_{cd,wd_mux} = 12 + 60 + 15 = 87 \text{ ps}$$

$$t_{ccq} + t_{cd} > t_{hold}$$

$$10 + 87 > 60$$

$$97 > 60$$

➔ Hold time ihlali yoktur. Çünkü $t_{ccq} + t_{cd} = 97 \text{ ps} > t_{hold} = 60 \text{ ps}$

➔ Dolayısıyla buffer eklenmesine gerek yoktur.

6. Sonuç:

Bu çalışmada 32-bitlik bir ALU tasarlanmış ve RegFile ile birleştirilerek tam bir datapath oluşturulmuştur. Tasarımda yüksek performans elde etmek amacıyla Carry Lookahead Adder tercih edilmiş, böylece kritik yol gecikmesi azaltılarak sistemin maksimum çalışma frekansı artırılmıştır. Yapılan zamanlama analizleri sonucunda setup ve hold koşullarının sağlandığı gösterilmiş, sistemin yaklaşık 1.25 GHz frekansta güvenilir şekilde çalışabileceği belirlenmiştir. Bu doğrultuda tasarımın hem fonksiyonel hem de zamanlama açısından doğru ve kararlı olduğu doğrulanmıştır.